

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА
ПОЛІТЕХНІКА”**

Кафедра систем штучного інтелекту



Проект
з дисципліни
«Видобування великих даних»

Виконали:
студенти групи ШІ-32
Воробець Максим
Лірко Максим
Полюга Юрій
Сидор Дмитро
Тимофеюк Марко
Яремчук Павло

Викладач:
Гентош Л. І.

Львів – 2026 р.

Мета:

- Набути навичок роботи із популярним фреймворком Apache Spark.
- Практика створення запитів до розподілених даних за допомогою SQL.
- Робота з Великими даними із відкритих джерел.
- Практичні навички роботи з популярними менеджерами сховища GitLab або Github.
- Набути досвіду командної роботи в галузі аналізу Великих даних.

Підготовчий етап

Для збереження коду та історії змін було обрано сервіс GitHub. Створено новий репозиторій з назвою `imdb-spark-project`. Налаштовано рівень доступу для викладача, щоб викладач мав вільний доступ до перевірки коду.

Для організації командної роботи та забезпечення стабільності коду було впроваджено модель розробки Gitflow:

- Гілка `main`: Ця гілка містить лише стабільний, протестований код, готовий до здачі.
- Гілка `develop`: Створена від гілки `main`. Вона слугує основною гілкою для інтеграції нових функцій.

Процес роботи: Вся розробка окремих модулів буде вестися в окремих "feature-гілках", які згодом будуть вливатися (Merge Request / Pull Request) у гілку `develop`. Це дозволяє проводити Code Review перед зміною основного коду.

Репозиторій було клоновано на локальну машину розробника за допомогою команди `git clone`. Проєкт відкрито у середовищі розробки (IDE) - PyCharm (або VS Code). Ініціалізовано віртуальне середовище (Virtual Environment - `venv`) для ізоляції залежностей проєкту.

Для запобігання потраплянню у віддалений репозиторій системних файлів, скомпільованого коду та великих файлів даних, було створено файл `.gitignore`. До нього додано наступні виключення:

- Папка віртуального оточення (`venv/`, `.env`).
- Кеш Python (`__pycache__` /, `*.pyc`).
- Налаштування IDE (`.idea/`, `.vscode/`).
- Логи та тимчасові файли Spark (`metastore_db/`, `*.log`).

Будь-які файли даних (`*.csv`, `*.tsv`, `*.gz`), щоб не перевантажувати репозиторій.

З офіційного джерела datasets.imdbws.com було завантажено архіви з даними.

Згідно з кращими практиками роботи з Big Data, файли датасету було розміщено у зовнішній папці на локальному диску, яка знаходиться поза кореневою директорією проєкту.

Мета: Це гарантує, що гігабайти даних випадково не будуть заіндексовані Git і не потраплять на GitHub, а код буде звертатися до них за абсолютним або відносним шляхом.

Ми зосередимось на обробці реляційних даних, що пов'язують фільми, їхні рейтинги та учасників зйомок.

Цільові файли:

- `title.basics.tsv.gz`: містить метадані (назва, рік, жанр).
- `title.ratings.tsv.gz`: містить числові метрики (рейтинг, кількість голосів).
- `title.principals.tsv.gz` та `title.crew.tsv.gz`: інформація про команду.

Аналітичні ознаки: `averageRating` та `numVotes` (для визначення популярності), `startYear` (для аналізу динаміки), `genres` (для категоризації), `runtimeMinutes`.

Ідентифікатори: `tconst` та `nconst` є ключовими для виконання операцій JOIN у Spark.

Ми вирішили ігнорувати файл `title.akas.tsv.gz` (локалізовані назви), оскільки для аналізу глобальних трендів достатньо оригінальної назви.

Також буде відкинуто записи з пустими значеннями (`\N`) у критично важливих полях (наприклад, де відсутній рік випуску або жанр), щоб уникнути викривлення статистики.

Для оцінки обчислювальних ресурсів, необхідних для обробки даних у Spark, було проаналізовано розмірність (кількість ознак) та фізичний обсяг даних.

Загальна кількість атрибутів у всіх таблицях становить 39 ознак (стовпців). Розподіл по файлах:

- `title.basics`: 9 ознак (включаючи `genres`, `runtimeMinutes`).
- `title.ratings`: 3 ознаки (критично важливі для Target variable).
- `title.principals`: 6 ознак (зв'язки персоналій з фільмами).
- `title.crew`: 3 ознаки.
- `title.akas`: 8 ознак (найбільша кількість текстових даних).

- name.basics: 6 ознак.
- title.episode: 4 ознаки.

Оскільки дані надаються у стиснутому форматі .gz, важливо розрізняти обсяг для зберігання та обсяг для обробки в оперативній пам'яті (In-Memory Processing).

Стиснутий обсяг (Storage Size):

- Загальний розмір архівних файлів становить приблизно ~1.2 ГБ – 1.5 ГБ (залежно від дати вивантаження).
- Це дозволяє легко зберігати "сирі" дані (Raw Data) локально.

Розархівований обсяг (Uncompressed / In-Memory Size):

- Після розпакування (що PySpark робить автоматично під час читання) реальний обсяг текстових даних (TSV) зростає приблизно в 5-7 разів і становить ~8 ГБ – 10 ГБ.
- Найбільшим файлом є title.akas.tsv (через велику кількість локалізацій назв), який займає значну частину цього обсягу.
- Файл title.principals.tsv також є масивним через велику кількість рядків (відображення "багато-до-одного" для акторів і знімальної групи).

Такий обсяг даних (10 ГБ у розгорнутому вигляді) є класичним прикладом "Small Big Data". Він занадто великий для комфортної обробки в Excel або Pandas на слабкому ПК (може викликати MemoryError), але ідеально підходить для демонстрації можливостей Apache Spark, який може обробляти ці дані частинами (partitions) навіть на одній машині з обмеженою RAM (наприклад, 8-16 ГБ).

Етап налаштування

Для забезпечення стабільної роботи з Apache Spark у межах проекту було розглянуто два підходи до налаштування робочого оточення: через локальну інсталяцію Python та за допомогою Docker.

Локальне налаштування (Python & PySpark)

Цей метод було обрано як основний для поточної розробки у середовищі PyCharm.

- **Встановлення інтерпретатора:** Використовується Python (рекомендована версія 3.8) [методичка].
- **Середовище розробки:** Проект розгорнуто в IDE PyCharm.
- **Ізоляція залежностей:** Створено віртуальне середовище (venv) для запобігання конфліктам бібліотек. Папку .venv/ додано до файлу .gitignore, щоб не перевантажувати репозиторій.
- **Налаштування PySpark та Java:** Встановлено Java 11 та виконано інсталяцію фреймворку за допомогою пакетного менеджера командою:

```
pip install pyspark
```

- **Перевірка:** Створено файл main.py [методичка], у якому ініціалізовано тестовий DataFrame. Успішне виконання методу .show() підтвердило коректність налаштування Spark-сесії.

Контейнеризація (Docker)

Для забезпечення відтворюваності коду на будь-якій машині передбачено можливість запуску через Docker:

- **Підготовка:** Встановлено Docker Desktop та перевірено працездатність команди docker --version [методичка].
- **Dockerfile:** У корінь директорії додано конфігураційний файл (Dockerfile) для створення образу [методичка].
- **Збірка образу:** Створення образу з тегом my-spark-img виконується командою:

```
docker build -t my-spark-img .
```

- **Запуск:** Запуск аналітичного додатка в ізольованому контейнері здійснюється командою:

```
docker run my-spark-img
```

Верифікація налаштувань

Для підтвердження готовності середовища у корені проекту створено файл **main.py**, який є точкою входу [методичка]. У файлі реалізовано базовий скрипт:

1. Ініціалізація `SparkSession`.
2. Створення тестового `DataFrame`.
3. Виклик методу `.show()`.

Успішне відображення таблиці в консолі підтвердило, що зв'язок між Python, Java та Spark встановлено коректно, і система готова до обробки реальних даних IMDB.

Етап видобування

Відповідно до завдань проекту, на цьому етапі було налаштовано процес зчитування "сирих" даних та їх перетворення у структуровані об'єкти для подальшого аналізу.

1. Створення схем для набору даних (*Воробець / Яремчук*)

Для забезпечення цілісності даних, уникнення помилок типізації та оптимізації роботи фреймворку PySpark, було прийнято рішення не використовувати автоматичне визначення типів (`inferSchema`), а явно задати структури для кожного з файлів. Цю логіку винесено в окремий модуль `schemas.py`.

Використовуючи класи з модуля `pyspark.sql.types` (а саме `StructType`, `StructField`, `StringType`, `IntegerType`, `FloatType`), було створено 7 схем:

1. `name_basics_schema`: для базової інформації про акторів та знімальну групу (включаючи роки життя та професію).
2. `title_akas_schema`: для альтернативних та регіональних назв тайтлів.
3. `title_basics_schema`: ключова схема для метаданих про фільми (типи, оригінальні назви, тривалість у хвилинах `runtimeMinutes` та масив жанрів `genres`).
4. `title_crew_schema`: для зв'язку тайтлів із режисерами та сценаристами.
5. `title_episode_schema`: для ідентифікації епізодів у межах серіалів.
6. `title_principals_schema`: для детальної інформації про ролі та персонажів.
7. `title_ratings_schema`: ключова схема для аналітики, що приводить рейтинг (`averageRating`) до типу `FloatType`, а кількість голосів (`numVotes`) — до `IntegerType`.

2. Ініціалізація та зчитування `DataFrame` (*Лірко / Полюга*)

Процес створення `DataFrame` реалізовано у модулі `extract.py`. Робота починається зі створення точки входу — `SparkSession` із назвою застосунку "IMDb DataFrames".

Для кожного з файлів даних (які зберігаються у форматі .tsv.gz) було застосовано метод зчитування `spark.read.csv()` із накладанням відповідних схем. Для коректного парсингу специфічного формату IMDb було використано наступні опції читання:

- `.option("sep", "\t")`: вказує фреймворку, що розділювачем колонок є символ табуляції (Tab-Separated Values).
- `.option("header", True)`: підтверджує, що перший рядок кожного файлу містить назви атрибутів і не повинен зчитуватися як дані.
- `.option("nullValue", "\\N")`: важливе налаштування конвеєра даних, яке дозволяє PySpark автоматично розпізнавати специфічний для датасету IMDb текстовий маркер відсутніх даних (\N) та конвертувати його у стандартне порожнє значення (null). Це значно спрощує подальший етап попередньої обробки.

3. Перевірка коректності зчитування (Сидор / Тимофеев)

Для перевірки успішного зчитування даних та первинного ознайомлення з їхньою структурою було створено окремий скрипт `verify_data.py`. Процес валідації включав наступні кроки:

1. Візуальний огляд та перевірка структур: За допомогою методів `.show(5)` та `.printSchema()` було виведено перші 5 рядків та структуру для кожного з 7 датафреймів. Це дозволило переконатися, що задані схеми застосувалися коректно (наприклад, цілочисельні типи для років), дані у стовпцях не змістилися, а специфічні значення успішно зчиталися.
2. Оцінка об'єму даних: Використано метод `.count()` для підрахунку загальної кількості записів (рядків) у кожному з табличних масивів.
3. Перевірка унікальності ідентифікаторів: За допомогою комбінації методів `.select().distinct().count()` було підраховано кількість унікальних ключів — `tconst` для фільмів (у таблицях `title_basics` та `title_ratings`) та `nconst` для персоналій.
4. Аналіз категоріальних ознак: Виведено перелік усіх унікальних значень для ключових текстових атрибутів, зокрема типів

відеоконтенту (titleType у title_basics_df) та категорій ролей персоналу (category у title_principals_df). Для цього використовувався виклик `.distinct().show()`.

5. Виявлення пропущених значень (Nulls): За допомогою агрегаційних функцій PySpark (`sum` та `col(...).isNull().cast("int")`) було динамічно пройдено по всіх колонках датафрейму `title_basics_df` та підраховано точну кількість порожніх значень (`null`) для кожної ознаки. Це заклало фундамент для наступного етапу очищення даних.